

KNN Algorithm

1. Introduction to Lazy Learning

- **Lazy Learning:** A type of machine learning method where the system **delays generalization** until it is asked to make predictions.
- It does **not build an explicit model** during training. Instead, it **stores training data** and uses it during testing/prediction.

Key Features:

- Training phase is **fast** (just storing data).
 - Prediction phase is **slow** (requires searching the dataset).
 - Examples:
 - **k-Nearest Neighbor (KNN)**
 - Case-Based Reasoning
-

2. What is k-Nearest Neighbor (KNN)?

- **Definition:** KNN is a **supervised lazy learning algorithm** used for **classification and regression**.
- It classifies a new data point based on the **majority vote** of its nearest neighbors in the feature space.

KNN is called a **lazy learner** because:

1. **No Training Phase (No Model Building):**
 - Unlike algorithms such as Decision Trees or Logistic Regression, KNN does **not build an explicit model** during training.
 - It simply **stores the training dataset**.
2. **Computation is Deferred Until Query Time:**
 - All the "work" (distance calculations, neighbor finding, majority voting) happens **only when a prediction is requested**.
 - That means the algorithm is "lazy" during training and "active" during prediction.
3. **High Prediction Cost:**
 - Since no generalization is done upfront, KNN must **scan the entire dataset** for each new prediction.
 - This makes it computationally expensive at test time.

Compare **Lazy Learners (like KNN)** vs **Eager Learners (like Decision Trees, Naïve Bayes, SVM, etc.)** so students can clearly see the difference.

Feature	Lazy Learner (e.g., KNN)	Eager Learner (e.g., Decision Tree, Naïve Bayes)
---------	--------------------------	--

Feature	Lazy Learner (e.g., KNN)	Eager Learner (e.g., Decision Tree, Naïve Bayes)
Training Phase	No real training → just stores data.	Builds a generalized model from training data.
Prediction Phase	Slow → must compute distances to all stored data.	Fast → uses pre-built model to predict quickly.
Memory Usage	High → keeps all training samples.	Lower → stores only the model, not all raw data.
Generalization	Happens at prediction time (late).	Happens at training time (early).
Computation Cost	Low at training, high at testing.	High at training, low at testing.
Example Algorithms	KNN, Case-Based Reasoning.	Decision Trees, Naïve Bayes, SVM, Neural Networks.

Quick Example for Students

- **Lazy Learner (KNN):**
Imagine you don't study during the semester but keep all your notes. During the exam, you **search your notes** to find the answer.
- **Eager Learner (Decision Tree):**
Imagine you **study in advance**, summarize concepts into a mind map (the model). During the exam, you **recall directly** from your summarized knowledge.

Key Point for Exams:

- KNN is called a **Lazy Learner** because it **defers generalization** until prediction time.
- Decision Trees / Naïve Bayes are **Eager Learners** because they **generalize during training** and are ready with a model for predictions.

3. Steps of KNN Algorithm

1. Choose the number of neighbors **k**.
2. Calculate the **distance** between the test data and all training samples (commonly **Euclidean distance**).

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

3. Select the **k nearest neighbors**.
4. Count the **class votes** among neighbors.
5. Assign the **most frequent class** as the prediction.

Visual Representation of KNN

Example: Classifying a Test Point with k=3

Feature Space:

• (Class A) ○ (Class B)

○

•

? ← Test Point

Case: k=3

→ Nearest neighbors: [○, ○, •]

→ Prediction = Class B (majority vote = 2)

Case Study Example: Iris Dataset

- Dataset: **Iris Flower Dataset** (150 samples, 3 classes: Setosa, Versicolor, Virginica).
- Features: Sepal length, Sepal width, Petal length, Petal width.
- Task: Classify a new flower based on its feature measurements using **KNN**.

KNN Example: Iris Dataset

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build KNN model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Predictions
y_pred = knn.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```

# KNN Example: Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build KNN model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Predictions
y_pred = knn.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

Sample Output:

yaml

Accuracy: 0.9777

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.95	0.97	21
2	0.95	1.00	0.97	15

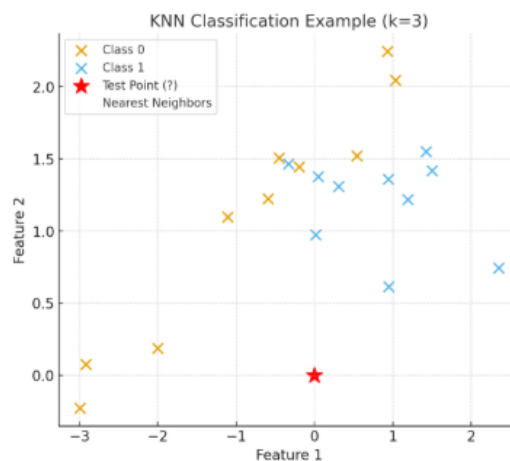
Advantages & Disadvantages of KNN

✔ Advantages

- Simple, intuitive, and effective.
- Works well with **low-dimensional data**.
- No training phase.

✗ Disadvantages

- Computationally expensive at prediction time.
- Sensitive to **irrelevant features & scaling**.
- Struggles with **large datasets**.



Here's a hand-drawn style visualization of KNN 🎨:

- The red star (?) is the test point.
- The circled points are the 3 nearest neighbors (k=3).
- The classes are shown in different colors/shapes.

This way, students can clearly see how KNN classifies based on the majority vote of nearby points.

Probabilistic Learning – Naive Bayes Classifier

1. What is Naive Bayes?

- **Naive Bayes (NB)** is a **probabilistic classifier** based on **Bayes' Theorem**.
- It is called "**naive**" because it assumes that all features are **independent** given the class label.
- Despite this strong assumption, it works **surprisingly well** in many applications, especially text classification.

2. Bayes' Theorem

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

Where:

- $P(C|X)$: Probability of class C given data X (posterior).
- $P(X|C)$: Probability of data X given class C (likelihood).
- $P(C)$: Prior probability of class C .
- $P(X)$: Overall probability of data X .

3. Types of Naive Bayes

- **Gaussian Naive Bayes** → for continuous features (assumes Gaussian distribution).
- **Multinomial Naive Bayes** → for word counts (text classification).
- **Bernoulli Naive Bayes** → for binary features (word present/absent).

4. Visual Representation (Diagram)

Imagine we have two classes: **Spam** and **Ham**.

Each word in an email contributes to the probability:

The diagram illustrates the process of calculating word probabilities for classification. It shows an email text "Win a free prize" with arrows pointing from each word to a comparison of probabilities: $P(\text{word} \mid \text{Spam})$ vs $P(\text{word} \mid \text{Ham})$. This represents how each word's likelihood is assessed for both classes.

We multiply these probabilities with the prior of Spam/Ham and choose the **class with the higher probability**.

5. Case Study: Spam Classification

- Dataset: SMS Spam Collection (spam vs ham).
- Features: Words in the message.
- Method: Use Multinomial Naive Bayes to calculate probability of words in Spam vs Ham.
- Example:
 - Email: "Win free money"
 - NB checks:
 - $P(\text{"Win"} \mid \text{Spam}) \times P(\text{"free"} \mid \text{Spam}) \times P(\text{"money"} \mid \text{Spam})$
 - vs. same for Ham
 - If Spam probability > Ham probability → classify as **Spam**.

• 6. Simple Python Code (PyCharm)

- `# Naive Bayes Example: Iris Dataset`
- `from sklearn.datasets import load_iris`
- `from sklearn.model_selection import train_test_split`
- `from sklearn.naive_bayes import GaussianNB`
- `from sklearn.metrics import accuracy_score`
-
- `# Load dataset`
- `X, y = load_iris(return_X_y=True)`
-
- `# Split dataset`
- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)`
-
- `# Create model`
- `model = GaussianNB()`
- `model.fit(X_train, y_train)`
-
- `# Predict`
- `y_pred = model.predict(X_test)`
-
- `# Evaluate`
- `print("Accuracy:", accuracy_score(y_test, y_pred))`

-  This code trains a **Gaussian Naive Bayes** model on the **Iris dataset**.

```
# Naive Bayes Example: Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create model
model = GaussianNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
```

7. Advantages & Disadvantages

Advantages

- Simple and fast.
- Works well with text data.
- Requires less training data.

Disadvantages

- Assumes features are independent (often not true).
- Struggles with correlated features.

Q1. Why is Naive Bayes called “naive”?

→ Because it assumes features are independent given the class label.

Q2. Mention two applications of Naive Bayes.

→ Spam filtering, sentiment analysis.

Q3. Which type of Naive Bayes is used for continuous data?

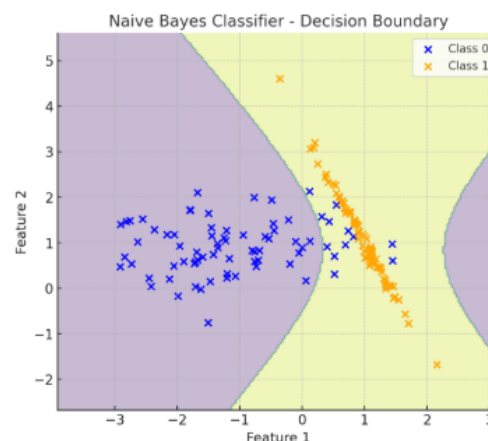
→ Gaussian Naive Bayes.

Q4. In text classification, why do we use smoothing?

→ To avoid zero probability for unseen words.

9. References

- Scikit-learn: Naive Bayes
- UCI SMS Spam Dataset: [Link](#)



Here's a hand-drawn style diagram of the Naive Bayes decision boundary:

- The **blue** and **orange** points are two different classes.
- The shaded regions show how **Naive Bayes separates the feature space** based on probability.
- The decision boundary is smooth, since Naive Bayes assumes a Gaussian distribution for features.

Review of probability - Joint probability, Conditional probability and Bayes' theorem

1. Review of Probability

1.1. Basic Definitions

- **Probability (P):** A measure of how likely an event is to occur.

$$0 \leq P(A) \leq 1, \quad P(\Omega) = 1$$

(Where Ω is the sample space.)

- **Random Experiment:** A process with uncertain outcomes (e.g., tossing a coin).
 - **Event:** A specific outcome (e.g., getting a Head).
-

1.2. Joint Probability

- The probability of **two events happening together**.

$$P(A \cap B) = P(A, B)$$

Example: Probability of rolling a 2 on dice **and** flipping a Head:

$$P(\text{Dice}=2, \text{Coin}=\text{Head}) = P(\text{Dice}=2) \times P(\text{Coin}=\text{Head}) = \frac{1}{6} \times \frac{1}{2} = \frac{1}{12}$$

1.3. Conditional Probability

- Probability of event A given that event B has already happened:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Example:

- $P(\text{Rain}|\text{Cloudy})$ = Probability it rains if we know it is cloudy.
- If $P(\text{Rain} \cap \text{Cloudy}) = 0.3$, $P(\text{Cloudy}) = 0.5$, then

$$P(\text{Rain}|\text{Cloudy}) = \frac{0.3}{0.5} = 0.6$$

1.4. Bayes' Theorem

- Relates conditional probabilities:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Interpretation:

- Prior $P(A)$: What we believed before seeing evidence.
- Likelihood $P(B|A)$: How well the evidence supports event A .
- Posterior $P(A|B)$: Updated probability after seeing evidence.

Example:

- 1% of people have a disease $\rightarrow P(\text{Disease}) = 0.01$.
- Test detects disease correctly 90% $\rightarrow P(\text{Positive}|\text{Disease}) = 0.9$.
- Test false positive rate 5% $\rightarrow P(\text{Positive}|\text{NoDisease}) = 0.05$.
- If a person tests positive:

$$P(\text{Disease}|\text{Positive}) = \frac{0.9 \times 0.01}{(0.9 \times 0.01) + (0.05 \times 0.99)} \approx 0.15$$

So even after a positive test, probability of disease is only 15%.

2. From Bayes' Theorem → Naive Bayes Classifier

- In classification, we want:

$$P(\text{Class}|\text{Features})$$

- By Bayes' Theorem:

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

- Since $P(X)$ is the same for all classes, we only compare:

$$\hat{C} = \arg \max_C P(X|C) P(C)$$

Naive Assumption:

$$P(X|C) = \prod_{j=1}^d P(x_j|C)$$

(Assumes features are independent given the class.)

3. Diagram (Flow of Probabilistic Learning)

```
Data → Calculate Priors (P(C))  
      → Calculate Likelihoods (P(x|C))  
      → Apply Bayes' Rule  
      → Choose class with highest Posterior (P(C|X))
```

Probabilistic Learning: Bayes' Theorem & Naive Bayes Classifier

1. Review of Probability

Before diving into Bayes' theorem, it's important to recall some probability concepts:

1.1 Joint Probability

- **Definition:** Probability of two events occurring together.
- **Formula:**

$$P(A \cap B) = P(A \text{ and } B)$$

- **Example:** Probability of drawing a red card **and** it being a king from a deck of 52 cards:

$$P(\text{Red} \cap \text{King}) = \frac{2}{52} = 0.0385$$

1.2 Conditional Probability

- **Definition:** Probability of an event occurring given that another event has occurred.
- **Formula:**

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) \neq 0$$

- **Example:** Probability of drawing a king given the card is red:

$$P(\text{King}|\text{Red}) = \frac{P(\text{Red} \cap \text{King})}{P(\text{Red})} = \frac{2/52}{26/52} = 0.0769$$

2. Bayes' Theorem

- **Purpose:** Allows us to update the probability of a hypothesis based on new evidence.
- **Formula:**

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}, \quad P(B) \neq 0$$

Where:

- $P(A|B)$ = Posterior probability (updated probability of A given B)
- $P(B|A)$ = Likelihood (probability of observing B given A)
- $P(A)$ = Prior probability of A
- $P(B)$ = Marginal probability of B

2.1 Example of Bayes' Theorem

Scenario: Medical test for a disease

- Probability of disease $P(D) = 0.01$
- Probability of no disease $P(\neg D) = 0.99$
- Test accuracy: $P(\text{Positive}|D) = 0.95$ (sensitivity)
- False positive: $P(\text{Positive}|\neg D) = 0.05$

Question: Probability that a person has the disease given a positive test?

$$P(D|\text{Positive}) = \frac{P(\text{Positive}|D) \cdot P(D)}{P(\text{Positive}|D) \cdot P(D) + P(\text{Positive}|\neg D) \cdot P(\neg D)}$$
$$P(D|\text{Positive}) = \frac{0.95 \cdot 0.01}{0.95 \cdot 0.01 + 0.05 \cdot 0.99} = 0.161$$

✓ Even with a positive test, only 16% chance of having the disease (shows importance of prior probability).

3. Naive Bayes Algorithm

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "naive" assumption that features are independent.

3.1 Formula

For a given class C_k and features $X = (x_1, x_2, \dots, x_n)$:

$$P(C_k|X) = \frac{P(C_k) \cdot \prod_{i=1}^n P(x_i|C_k)}{P(X)}$$

- **Prediction:** Choose class with highest posterior probability:

$$\hat{C} = \arg \max_{C_k} P(C_k) \cdot \prod_{i=1}^n P(x_i|C_k)$$

3.2 Types of Naive Bayes

1. **Gaussian NB:** For continuous features (assumes normal distribution).
2. **Multinomial NB:** For discrete features (e.g., word counts in text).
3. **Bernoulli NB:** For binary features (presence/absence of attributes).

3.3 Example (Text Classification)

Suppose we classify emails as **Spam** or **Not Spam** using two words: **offer** and **win**.

Email	Offer	Win	Class
1	Yes	Yes	Spam
2	No	Yes	Spam
3	Yes	No	Not Spam
4	No	No	Not Spam

Step 1: Calculate prior probabilities:

$$P(\text{Spam}) = \frac{2}{4} = 0.5, \quad P(\text{Not Spam}) = 0.5$$

Step 2: Calculate likelihoods (e.g., $P(\text{Offer}=\text{Yes}|\text{Spam}) = \frac{1}{2} = 0.5$)

Step 3: Use Naive Bayes formula to predict a new email: **offer=Yes**, **win=Yes**.

3.4 Python Code Example

```
# Sample Naive Bayes Implementation
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

```

from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[1,1],[0,1],[1,0],[0,0]] # Features: Offer, Win (1=Yes, 0=No)
y = [1,1,0,0] # Labels: 1=Spam, 0=Not Spam

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train Gaussian Naive Bayes
model = GaussianNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Predictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

```

# Sample Naive Bayes Implementation
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Sample Dataset
X = [[1,1],[0,1],[1,0],[0,0]] # Features: Offer, Win (1=Yes, 0=No)
y = [1,1,0,0] # Labels: 1=Spam, 0=Not Spam

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Train Gaussian Naive Bayes
model = GaussianNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Predictions:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

3.5 Advantages

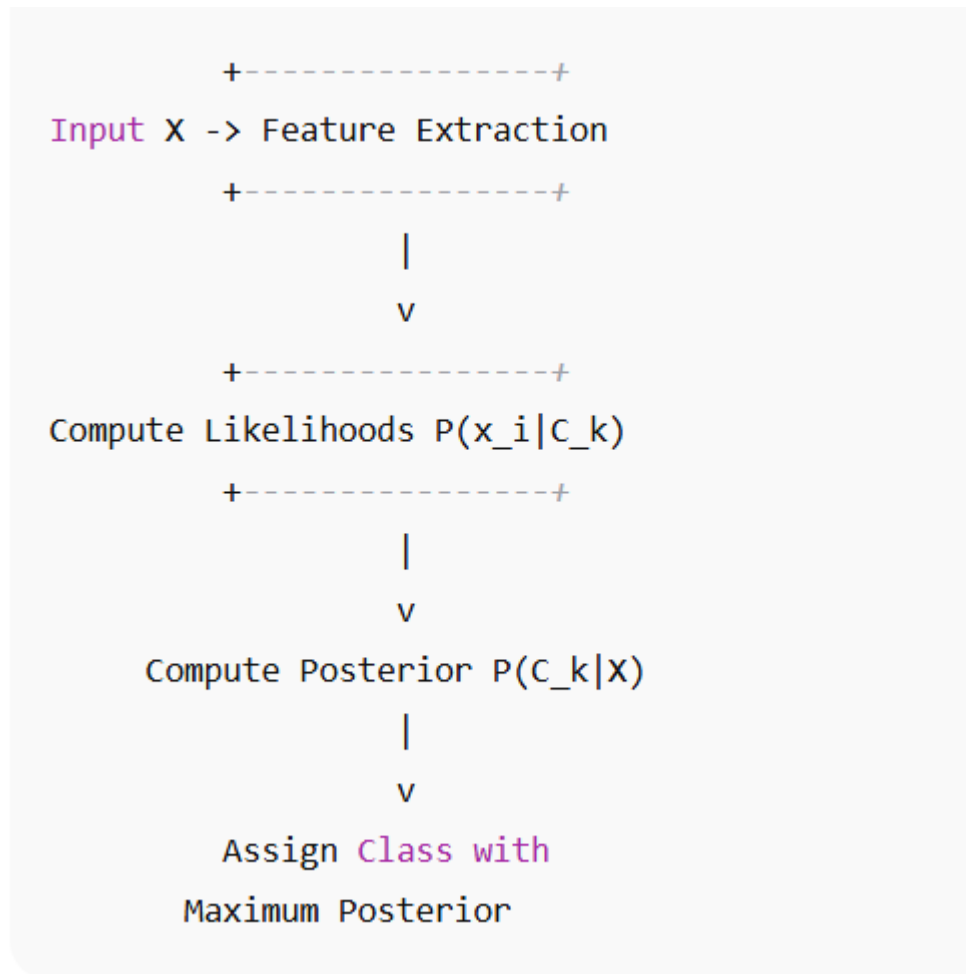
- Simple and fast.
- Performs well with high-dimensional data (like text).
- Handles both continuous and discrete features.

3.6 Limitations

- Assumes independence between features (rarely true in real life).
- Poor performance if features are highly correlated.

4. Visual Diagram

Naive Bayes Classifier Workflow



5. Key Points to Remember

1. Bayes' theorem updates probabilities based on new evidence.
2. Naive Bayes assumes feature independence for simplicity.
3. Works best for text classification, spam detection, sentiment analysis.
4. Simple, fast, but independence assumption can limit accuracy.

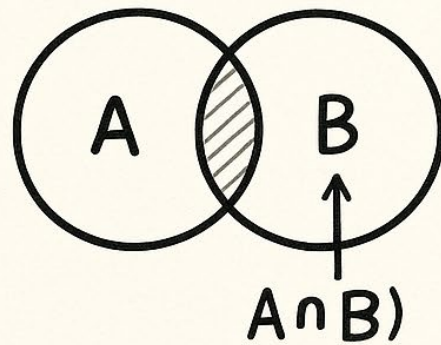
BAYES' THEOREM & NAIVE BAYES

BAYES' THEOREM

$$P(A|B) = \frac{P(B|A)}{P(B)}$$

Posterior

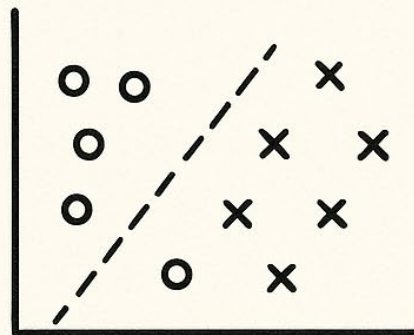
Likelihood



NAIVE BAYES

$$P(C_k | X) = \frac{P(C_k)}{P(x_i | C_k)}$$

FEATURE SPACE

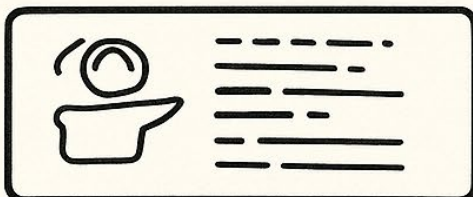


EXAMPLE



OFFER	WIN	CLASS
Yes	No	Not Spam
No	Yes	Spam

PYTHON CODE



```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)
```